MLDLS · CHAPTER 01 · WORKSHOP 2

Scikit-learn and the ML Workflow

Discovering the Scikit-learn library and running the eight steps of a machine-learning project on one example

Module	Machine Learning, Deep Learning and Security (UEM7 · 4 credits · coefficient 2)
Specialty / Semester	Computer Security (Sécurité Informatique) · Semester 07
Chapter	Chapter 01 — Machine Learning
Session type	Workshop (TP) 2 of 3 · hands-on, individual or pairs
Duration	1 session of 1 h 30
Tools	Python 3, Jupyter / Colab, Scikit-learn, Pandas, NumPy, Matplotlib, joblib
Instructor	Dr. Mohamed SANDELI, Dr. Wafa MOUSSER.
Academic year	2026 – 2027

1. Overview

Workshop 1 taught the data tools. This workshop introduces **Scikit-learn**, the reference Python library for classical machine learning, and uses it to run the complete workflow of Chapter 1 §1 — define the problem, load, explore, prepare, choose a model, train, evaluate, tune, save — on one classification example. The same code pattern (`fit`, `predict`, `score`) is then reused for a regression example in ten lines, to show that the workflow does not depend on the algorithm.

The example dataset is the **Wisconsin breast-cancer** dataset shipped with Scikit-learn (569 tumours described by 30 measurements, benign or malignant). It needs no download and has a clear "positive class" whose misses are costly — the same situation as an intrusion or a malware sample in Workshop 3.

1.1 Learning outcomes

1. Describe what Scikit-learn provides and use its uniform estimator interface: `fit`, `predict`, `predict_proba`, `score`, `transform`.
2. Run the eight steps of the ML workflow on a dataset, naming the Scikit-learn function used at each step.
3. Split data into training and test sets, scale features correctly, and explain why the test set must stay untouched.
4. Read a confusion matrix and compute accuracy, precision and recall; move the decision threshold.
5. Compare two or three algorithms quickly with cross-validation, save a trained model and use it on a new observation.

1.2 Prerequisites

- Workshop 1 (arrays, DataFrames, basic plots).

- Chapter 1 lecture, Sections 1 (workflow), 3.5 and 4.4 (the Scikit-learn snippets) and the metrics tables.
- `import sklearn` works — otherwise `!pip install scikit-learn`.

1.3 Timetable (1 h 30)

Time	Part	Content
0:00 – 0:10	Part 1	What Scikit-learn is; the estimator interface; the workflow mapped to functions
0:10 – 1:05	Part 2	The eight steps on the breast-cancer dataset (classification)
1:05 – 1:15	Part 3	Same workflow, regression version, in ten lines
1:15 – 1:30	Wrap-up	Exercise, checkpoint questions, homework

2. Part 1 — Scikit-learn in ten minutes

Scikit-learn (`sklearn`) contains the classical algorithms of the lecture — linear and logistic regression, decision trees, random forests, naive Bayes, SVM, K-nearest neighbours, K-Means — plus everything around them: splitting, scaling, metrics, cross-validation, pipelines. Every algorithm is an **estimator** with the same four methods, so switching models means changing one line.

Method	Meaning	Lecture step
`model = Algorithm(hyper_parameters)`	Choose the model family and its settings	Step 1 — define the model
`model.fit(X_train, y_train)`	Learn the parameters Θ from the training data	Steps 2–5 — initialise, predict, cost, gradient descent
`model.predict(X_new)`	Predict a class (classifier) or a number (regressor)	Use the model
`model.predict_proba(X_new)`	Class probabilities Ŷ (classifiers only)	The sigmoid output of §4.1
`model.score(X_test, y_test)`	Default metric: accuracy (classifier) or R^2 (regressor)	Step 6 — evaluate
`scaler.fit_transform(X_train)`, `scaler.transform(X_test)`	Learn a preprocessing on train, apply it to both	Data preparation

2.1 The workflow of Chapter 1 §1, mapped to Scikit-learn

#	Workflow step	Scikit-learn / Pandas tool
1	Define the problem: input X, output Y, number or category?	— (thinking, not code)
2	Collect and explore the data	`pd.DataFrame`, `describe`, `value_counts`, plots
3	Prepare: clean, encode, split, scale	`train_test_split`, `StandardScaler`, `OneHotEncoder`
4	Select a model family	`LogisticRegression()`, `DecisionTreeClassifier()`, …

#	Workflow step	Scikit-learn / Pandas tool
5	Train on the training set	`model.fit(X_train, y_train)`
6	Evaluate on the test set with the right metric	`confusion_matrix`, `precision_score`, `recall_score`, `r2_score`
7	Tune hyper-parameters, threshold, compare models	`cross_val_score`, `GridSearchCV`, `predict_proba`
8	Deploy and monitor	`joblib.dump` / `joblib.load`, a prediction function

Cell 1.1

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import sklearn
print(sklearn.__version__)
```

3. Part 2 — The eight steps on one example (55 min)

Step 1 — Define the problem

Input X: 30 measurements of a tumour (radius, texture, area…). Output Y: benign or malignant — a **category**, so this is **classification** (Chapter 1 §4). The class we want to catch is *malignant*; it will be coded 1 and called the **positive class**. Missing a malignant tumour (false negative) is worse than a false alarm (false positive): keep this in mind for Step 7.

Step 2 — Load and explore

Cell 2.1

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(1 - data.target, name="malignant")      # sklearn codes benign = 1 ; we want malignant = 1


print(X.shape)                       # (569, 30)
print(y.value_counts())              # 357 benign (0), 212 malignant (1)
X.iloc[:, :5].describe().round(1)    # the first five features : very different scales !
X.groupby(y).mean().iloc[:, :5].round(1)   # do malignant tumours look different on average ?
```

Two observations that drive the next steps: the classes are unbalanced (37 % positive), so accuracy alone will not be enough; and the features live on very different scales (`mean area` ≈ 650, `mean smoothness` ≈ 0.1), so scaling is required.

Step 3 — Prepare: split then scale

Cell 2.2

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)   # 426 train / 143 test, same class ratio
print(X_train.shape, X_test.shape)


scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # learn mean and std on TRAIN, then transform train
X_test_s  = scaler.transform(X_test)        # apply the SAME mean and std to test
```

> **The golden rule**
> The test set simulates the future: data the model has never seen. Anything learned from data — the scaler's mean and std, the model's weights — is learned on the training set only. `fit_transform` on train, `transform` on test. Fitting the scaler on the whole dataset is called **data leakage** and makes the test score optimistic.

Step 4 and 5 — Choose a model and train it

Cell 2.3

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)   # Step 4 : the model of Chapter 1 §4.1
model.fit(X_train_s, y_train)               # Step 5 : gradient descent happens here
print("accuracy on train:", round(model.score(X_train_s, y_train), 3))   # 0.988
print("accuracy on test :", round(model.score(X_test_s,  y_test), 3))    # 0.965
```

Train accuracy is slightly higher than test accuracy, which is normal; a large gap would indicate overfitting (Chapter 1 §2). The learned weights are in `model.coef_` (one per feature) and `model.intercept_` (the bias Θ_0).

Step 6 — Evaluate with the right metrics

Cell 2.4

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, precision_score, recall_score, f1_score

y_pred = model.predict(X_test_s)
cm = confusion_matrix(y_test, y_pred)
print(cm)                                   # [[TN FP]
                                            #  [FN TP]]
ConfusionMatrixDisplay(cm, display_labels=["benign", "malignant"]).plot(); plt.show()

print("precision:", round(precision_score(y_test, y_pred), 3))   # of the alerts, how many are right ?
print("recall   :", round(recall_score(y_test, y_pred), 3))      # of the malignant, how many are caught ?
print("F1       :", round(f1_score(y_test, y_pred), 3))
```

Expected: [[89, 1], [4, 49]] — precision 0.98, recall 0.925, F1 0.951. Four malignant tumours out of 53 are missed.

Metric	Formula	Reads as	When it matters
Accuracy	(TP + TN) / all	"right 96.5 % of the time"	Only when classes are balanced and errors cost the same
Precision	TP / (TP + FP)	"98 % of the alerts are real"	When false alarms are expensive (analyst time)
Recall	TP / (TP + FN)	"92.5 % of the real cases are caught"	When a miss is expensive (disease, intrusion)
F1	2·P·R / (P + R)	balance of the two	To compare models with one number

Step 7 — Tune: the threshold, then compare algorithms

Cell 2.5 — move the threshold

```
y_prob = model.predict_proba(X_test_s)[:, 1]          # P(malignant), the sigmoid output
for t in [0.3, 0.5, 0.7]:
    yp = (y_prob >= t).astype(int)                       # the decision rule of §4.1 with a movable threshold
    tn, fp, fn, tp = confusion_matrix(y_test, yp).ravel()
    print(f"threshold {t}: missed malignant (FN) = {fn}, false alarms (FP) = {fp}")
```

Expected: t = 0.3 → FN 2, FP 1 · t = 0.5 → FN 4, FP 1 · t = 0.7 → FN 5, FP 0. Lowering the threshold catches more positives at the price of more false alarms — the trade-off mentioned in §4.1.

Cell 2.6 — compare algorithms with cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier


candidates = {"logistic regression": LogisticRegression(max_iter=1000),
              "decision tree":       DecisionTreeClassifier(random_state=42),
              "k-nearest neighbours": KNeighborsClassifier(5),
              "random forest":       RandomForestClassifier(random_state=42)}


for name, algo in candidates.items():
    pipe = make_pipeline(StandardScaler(), algo)      # scaler + model in one object : no leakage
    scores = cross_val_score(pipe, X_train, y_train, cv=5)   # 5 train/validation splits of the TRAIN set
    print(f"{name:22s} mean accuracy = {scores.mean():.3f}")
```

Expected: logistic 0.977, random forest 0.965, K-NN 0.962, decision tree 0.908. Cross-validation uses only the training set, so the test set is still untouched for the final check.

A **pipeline** chains the scaler and the model: `pipe.fit(X_train, y_train)` scales then trains, `pipe.predict(X_new)` scales then predicts. It is the simplest way to avoid leakage and the object we will save.

Step 8 — Save, reload, use on a new case

Cell 2.7

```
import joblib
final = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_train, y_train)
print("final test accuracy:", round(final.score(X_test, y_test), 3))


joblib.dump(final, "tumor_model.joblib")            # the file you would ship
loaded = joblib.load("tumor_model.joblib")


new_case = X_test.iloc[[0]]                          # one new observation (a 1-row DataFrame)
print(loaded.predict(new_case), loaded.predict_proba(new_case).round(3), "truth:", int(y_test.iloc[0]))
```

> **Deployment in one sentence**
>
> A deployed model is this `.joblib` file plus a function that checks the input (same 30 columns, same order), calls `predict_proba`, applies the chosen threshold and returns an alert — and someone who watches whether the data and the results drift over time (Chapter 1 §1, Step 8).

4. Part 3 — The same workflow for regression (10 min)

Change the problem (predict a **number**: the response time of a web server, §3.4), change the model (`LinearRegression`), change the metric (RMSE, R^2). The eight steps and the code pattern stay identical.

Cell 3.1

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score


rng = np.random.default_rng(0); n = 300                                  # Step 2 : a synthetic log
users = rng.integers(50, 1200, n)
cpu   = np.clip(0.06 * users + rng.normal(0, 8, n), 2, 100).round(1)
resp  = (40 + 0.25 * users + 1.5 * cpu + rng.normal(0, 25, n)).round(0)
srv = pd.DataFrame({"users": users, "cpu": cpu, "response_ms": resp})


Xr, yr = srv[["users", "cpu"]], srv["response_ms"]                       # Step 1 : X and Y (a number)
Xr_train, Xr_test, yr_train, yr_test = train_test_split(Xr, yr, test_size=0.2, random_state=42)  # Step 3
reg = LinearRegression().fit(Xr_train, yr_train)                          # Steps 4-5
pred = reg.predict(Xr_test)
print("RMSE:", np.sqrt(mean_squared_error(yr_test, pred)).round(1), "ms")  # Step 6
print("R2  :", round(r2_score(yr_test, pred), 3))
print("equation: response =", reg.intercept_.round(1), "+", reg.coef_.round(3), "x [users, cpu]")
print(reg.predict(pd.DataFrame([[800, 60]], columns=["users", "cpu"])).round(0), "ms for 800 users at 60 % CPU")
```

Expected: RMSE ≈ 21 ms, R^2 ≈ 0.97, equation ≈ 37 + 0.242 × users + 1.66 × cpu (the generating law was 40 + 0.25 × users + 1.5 × cpu), prediction ≈ 331 ms.

	Classification (Part 2)	Regression (Part 3)
Output Y	a category (0 / 1)	a number (ms)
Model	`LogisticRegression`	`LinearRegression`
Prediction	`predict` (class), `predict_proba` (probability)	`predict` (number)
Metrics	confusion matrix, precision, recall, F1	RMSE, MAE, R^2
Everything else	split, scale, fit, cross-validate, save: identical	identical

5. Wrap-up, exercise and homework (15 min)

Exercise (10 min)

1. In Cell 2.6, add `GaussianNB()` (naive Bayes, `from sklearn.naive_bayes import GaussianNB`) and `SVC()` (`from sklearn.svm import SVC`) to the candidates and rerun. Which algorithm is best on this data?

2. Retrain the logistic regression **without** the scaler (`LogisticRegression(max_iter=1000).fit(X_train, y_train)`). Compare the test accuracy with Cell 2.3 and explain the difference in one sentence.

3. A hospital says a missed malignant tumour costs 20 times more than a false alarm. Which threshold of Cell 2.5 would you choose, and why?

Checkpoint questions

Q1. In which order: split or scale? Why?

Q2. Your model has accuracy 0.99 on a dataset where 1 % of the rows are attacks. Is it a good model? Which metric do you look at next?

Q3. What is the difference between `predict` and `predict_proba`?

Q4. Give the regression equivalents of `LogisticRegression`, `accuracy` and `confusion_matrix`.

5.1 Homework (for Workshop 3)

- Apply the eight steps to `logins_clean.csv` from Workshop 1: define the target `suspicious = failed_attempts >= 5`, use `hour`, `duration_min`, `night` and the one-hot encoded `country` (`pd.get_dummies`) as features, train a logistic regression and a decision tree, report the confusion matrix and recall on the test set. Expect a weak model — the data is random — and say so: the point is the procedure.
- Submit `TP2_<name>.ipynb` executed end to end, with the answers to the exercise and the checkpoint questions.

5.2 Assessment (/10)

Criterion	Points
Notebook runs end to end; split before scale; no leakage	3
Confusion matrix, precision and recall correctly read; threshold question answered	3
Exercise: algorithms compared, effect of scaling explained	2
Homework on the login log: procedure respected, limits stated	2